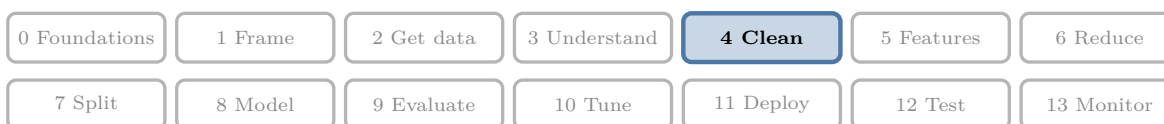


Handling Missing Data: Iterative Imputer (MICE)

Contents

1. Overview	2
2. When to use MICE	3
3. Advantages and disadvantages	4
4. The example table	4
5. Step 0: fill with the column means	5
6. Iteration 1: one feature at a time	5
6.1 The R&D column	6
6.2 The Administration column	6
6.3 The Marketing column	7
7. Repeating until the fills settle	7
7.1 The change between iterations	7
7.2 Why one iteration is not enough	7
7.3 When to stop	8
8. The iterative imputer in scikit-learn	9
8.1 Switching on the experimental class	9
8.2 The main parameters	9
8.3 Reproducing the hand calculation	10
8.4 Fitting on the training set only	11
8.5 Several imputations	12
9. Summary	13
10. Sources	13
11. Key terms	13

Where this fits: Pipeline map step 4 (Clean). See the [Course map](#).



- **Builds on:** Simple imputation (mean, median, mode, constant) ([Note 37](#)); Missing values ([Note 38](#)).
- **Compare with:** KNN imputer ([Note 39](#)).

1. Overview

Key point: MICE fills every gap with its feature's mean first, then improves the fills one feature at a time with a model trained on the other features, and repeats until the fills stop changing.

A **feature** is an input variable (one column of the data table), and an **observation** is one record (one row). The **target** is the output we predict.

The KNN imputer ([Note 39](#)) was the first multivariate technique: it fills a gap from the observations most similar to the one with the gap. This Note covers the second one, the **iterative imputer**. The iterative imputer turns each feature with gaps into a small prediction problem: that feature is the output, the other features are the inputs. Think of a crossword: each answer you write in gives letters that help with the crossing answers, so you go round the grid several times, fixing earlier guesses as the crossings fill in.

The algorithm behind it is **MICE**, short for **Multivariate Imputation by Chained Equations**. Figure 1 shows the whole loop:

- **Step 0:** fill every gap with the mean of its feature.
- **One iteration:** for each feature in turn, put its gaps back to NaN, train a model on the observations without a gap, and predict the gaps.
- **Stop** when the fills hardly change from one iteration to the next.

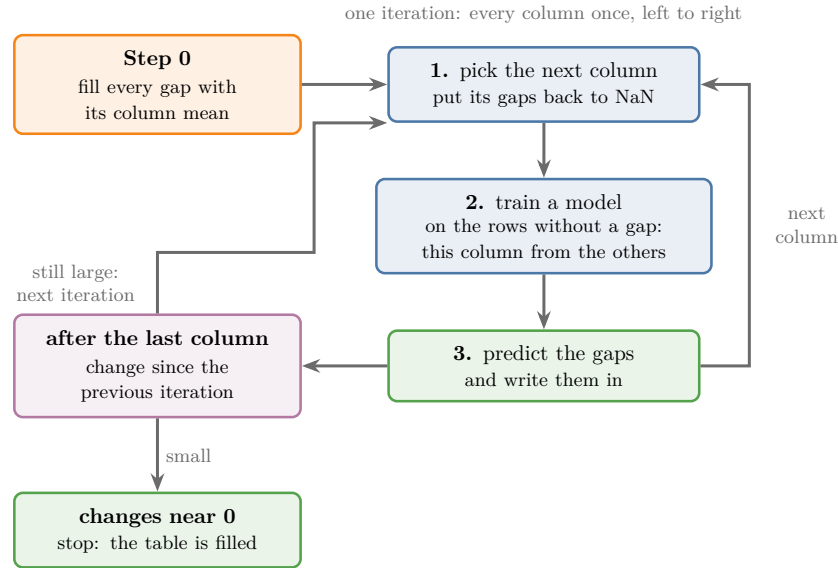


Figure 1: The MICE loop: a mean fill, then one model per column, repeated until the fills settle

Each model is one “equation” that predicts one feature. The equations are “chained” because each one uses the latest fills of the others.

2. When to use MICE

Key point: MICE suits data that is missing at random (MAR): the gaps can be predicted from the other features.

Note 35 (Section 5) names three ways in which data goes missing:

- **MCAR (missing completely at random):** the value was never collected, for no reason related to the data.
- **MAR (missing at random):** whether a value is missing depends on other features we can see. For example, older people skip an income question more often, and age is recorded. The other features carry information about the missing value.
- **MNAR (missing not at random):** whether a value is missing depends on the value itself, for example high earners hiding their income. The other features cannot fully account for the gaps.

MICE predicts a missing value from the other features, so it needs those features to be related to it. Imputation methods such as MICE commonly assume MAR, and checking that MAR is plausible is the first step of a MICE analysis (van Buuren and Groothuis-Oudshoorn 2011, §3.1 and §6.2). We can run MICE on any data, but it gives its best results under MAR.

3. Advantages and disadvantages

Key point: MICE is usually more accurate than simpler imputers, but it is slow and its fitted models must be kept for production.

Advantage:

- **Accurate.** The fills come from a model that uses every other feature, so they are usually closer to the truth than mean or median fills. Section 8.4 measures the gap on real data.

Disadvantages:

1. **Slow.** Every iteration trains one model per feature with gaps, and we run several iterations. On large data this takes time.
2. **More to keep in production.** A new observation with a gap must be filled the same way, so the fitted models are kept on the server: one model per feature and iteration, in the attribute `imputation_sequence_` (scikit-learn docs, `IterativeImputer`). Filling a new observation means running those models again. Mean imputation keeps only one number per feature.

4. The example table

Key point: Five startups, three spend features, one gap in each.

The data is the “50 Startups” dataset: for 50 companies, the money spent on R&D, administration and marketing, and the profit. We keep only the three spend features, because imputation is done on the inputs, not on the target `Profit`.

To follow every step by hand, we:

1. Divide all amounts by 10,000 dollars and round to whole numbers.
2. Keep 5 random observations.
3. Hide one value in each feature.

Row	R&D	Administration	Marketing
1	8	15	30
2	NaN (4)	5	20
3	15	10	41
4	12	NaN (16)	26
5	2	15	NaN (3)

The numbers in brackets are the hidden true values. The algorithm never sees them; we use them at the end to judge the fills.

5. Step 0: fill with the column means

Key point: The first guess for every gap is the mean of its feature: 9.25, 11.25 and 29.25.

MICE needs a complete table to train its first models, so it starts with the simplest fill (Note 36):

- **R&D:** $(8 + 15 + 12 + 2)/4 = 9.25$
- **Administration:** $(15 + 5 + 10 + 15)/4 = 11.25$
- **Marketing:** $(30 + 20 + 41 + 26)/4 = 29.25$

The mean-filled table is **iteration 0**. The mean ignores the other features, so these fills are rough starting points that the next steps improve.

6. Iteration 1: one feature at a time

Key point: For each feature, left to right: put its gap back to NaN, train a linear regression on the other four observations, predict the gap.

Figure 2 runs the whole process on the table. Iteration 1 goes through the three features in order.

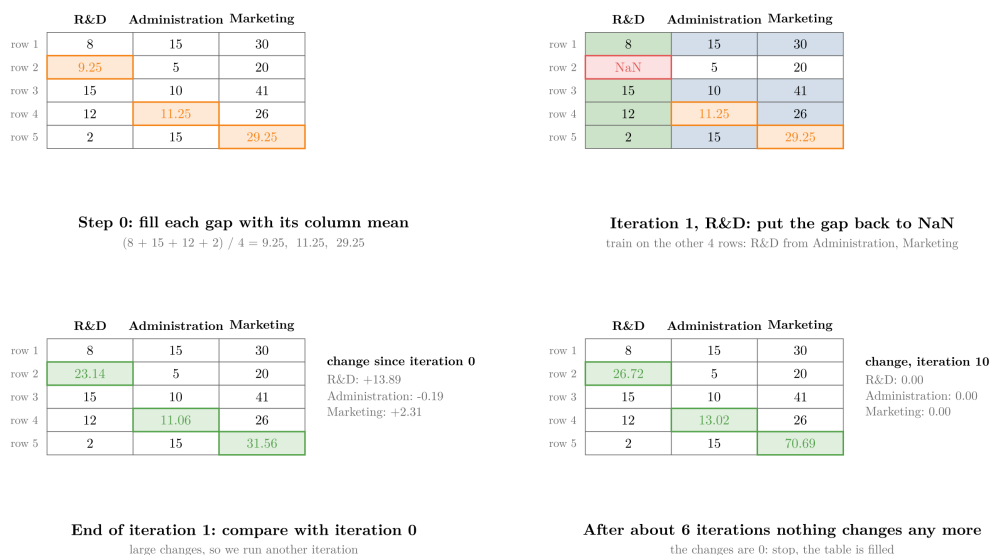


Figure 2: MICE on the 5-row table: mean fill, iteration 1 column by column, the change after each iteration, and the settled values

Any regression model can do the predicting: linear regression, a decision tree, a random forest. Here we use linear regression (Note 50). Values are kept to two decimals, as in a hand calculation.

6.1 The R&D column

Key point: R&D of row 2 is predicted from its Administration (5) and Marketing (20): 23.14.

1. Put the R&D gap of row 2 back to NaN. The mean fills in the other features (11.25, 29.25) stay.
2. The four observations with an R&D value become training data. Inputs: Administration and Marketing. Output: R&D.

Row	Administration (input)	Marketing (input)	R&D (output)
1	15	30	8
3	10	41	15
4	11.25	26	12
5	15	29.25	2

3. Train a linear regression on these four observations and predict R&D for row 2.

The prediction follows the usual three steps:

1. **In words:** R&D is a starting value plus a weight times Administration plus a weight times Marketing.
2. **Formula:** the trained model is

$$\text{R\&D} = 31.21 - 1.875 \times \text{Admin} + 0.065 \times \text{Marketing}$$

3. **Example:** row 2 has Administration 5 and Marketing 20:

$$31.21 - 1.875 \times 5 + 0.065 \times 20 = 31.21 - 9.37 + 1.30 = 23.14$$

The gap now holds 23.14 instead of the mean 9.25.

6.2 The Administration column

Key point: Administration of row 4 is predicted from R&D and Marketing, using the new 23.14: 11.06.

Put the Administration gap of row 4 back to NaN. The training rows are rows 1, 2, 3 and 5, with R&D and Marketing as inputs:

Row	R&D (input)	Marketing (input)	Administration (output)
1	8	30	15
2	23.14	20	5
3	15	41	10
5	2	29.25	15

Row 2 already uses its new R&D fill, 23.14. The model is $\text{Admin} = 15.65 - 0.491 \times \text{R\&D} + 0.050 \times \text{Marketing}$, and row 4 (R&D 12, Marketing 26) gives $15.65 - 5.89 + 1.30 = 11.06$.

6.3 The Marketing column

Key point: Marketing of row 5 is predicted from R&D and Administration: 31.56.

Put the Marketing gap of row 5 back to NaN. The training rows are rows 1 to 4, which now include both new fills, 23.14 and 11.06.

The model is $\text{Marketing} = 8.12 + 0.386 \times \text{R\&D} + 1.511 \times \text{Admin}$. Row 5 (R&D 2, Administration 15) gives $8.12 + 0.77 + 22.67 = 31.56$.

After the last feature, iteration 1 is complete and the table has no gaps:

Gap	Iteration 0 (mean)	Iteration 1
R&D, row 2	9.25	23.14
Administration, row 4	11.25	11.06
Marketing, row 5	29.25	31.56

7. Repeating until the fills settle

Key point: We subtract each iteration's table from the previous one and repeat until every change is close to 0.

7.1 The change between iterations

Key point: After iteration 1, the changes are +13.89, -0.19 and +2.31: still large, so we continue.

Subtracting the iteration-0 table from the iteration-1 table gives 0 for every known value, since those never change. Only the three gaps show a change:

- R&D: $23.14 - 9.25 = 13.89$
- Administration: $11.06 - 11.25 = -0.19$
- Marketing: $31.56 - 29.25 = 2.31$

These differences measure how far the regression fills moved away from the mean fills. While they are large, the fills are still moving.

7.2 Why one iteration is not enough

Key point: The first models were trained on mean fills, which may be wrong; each new iteration trains on better fills and gets closer to a stable answer.

In iteration 1, the R&D model was trained with Administration 11.25 and Marketing 29.25 in its rows: the rough mean fills. Its prediction inherits that error.

Iteration 2 starts from the iteration-1 table and repeats the same three steps: R&D, then Administration, then Marketing. Every model now trains on better fills than before.

Gap	Iteration 1	Iteration 2	Change
R&D, row 2	23.14	23.78	+0.64
Administration, row 4	11.06	11.22	+0.16
Marketing, row 5	31.56	38.87	+7.31

The R&D and Administration changes shrank. The Marketing change grew: the fills can swing up and down for a few iterations before they settle.

7.3 When to stop

Key point: Stop when all changes are close to 0, or after a fixed number of iterations such as 10.

Two stopping rules are common:

1. **Changes below a threshold:** stop when every fill moves by less than a small amount between two iterations.
2. **A fixed number of iterations:** for example 5, 10 or 20.

Figure 3 continues the example for 10 iterations at full precision. With linear regression (blue), the fills settle after about 6 iterations at 26.72, 13.02 and 70.69.

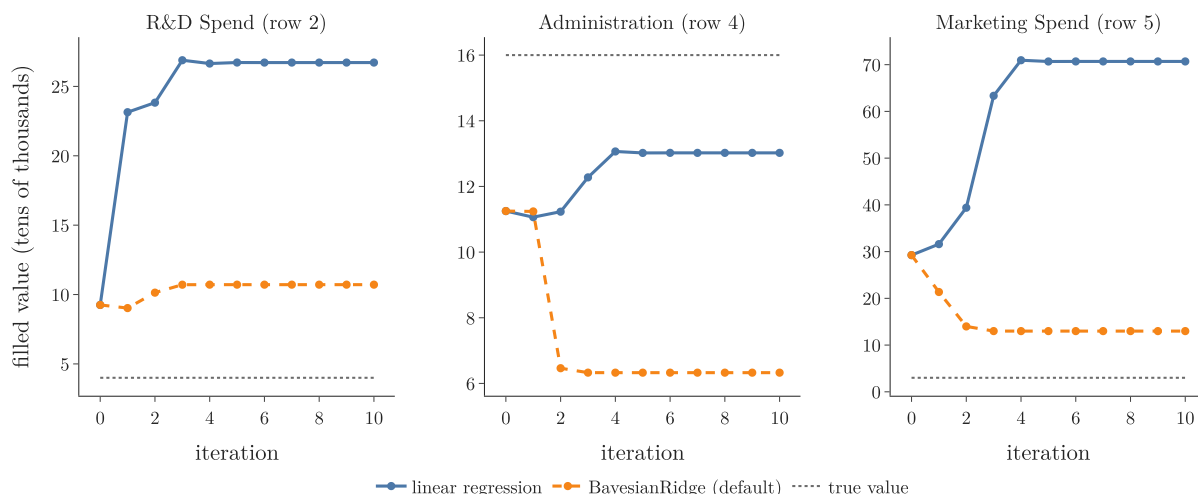


Figure 3: The three fills after each iteration, with linear regression and with scikit-learn's default BayesianRidge; the dotted line is the true hidden value

Extra: Keeping only two decimals at each step makes the hand-calculated numbers drift slightly from the full-precision ones. Iteration 1 gives 31.60 for Marketing instead of 31.56, and iteration 2 gives 23.83, 11.23 and 39.39. The final values are the same.

Extra: Settled is not the same as correct. The true values are 4, 16 and 3, and the linear-regression fills end far from them. Each model learns from only four observations and two inputs. At the settled values, all three linear models fit their four training observations exactly: every residual is 0.00. Two of the gaps are also predicted from inputs outside the training range: the R&D gap uses Administration 5, below the training values 10 to 15, and the Marketing gap uses R&D 2, below the training values 8 to 26.72. A linear model then simply extends its plane, far beyond any value it was trained on (70.69 for a feature whose known values run from 20 to 41). With scikit-learn's default model, `BayesianRidge` (orange), the fills settle at 10.71, 6.33 and 12.99: closer for R&D and Marketing. On real data with more observations, the fills come much closer to the truth (Section 8.4).

8. The iterative imputer in scikit-learn

Key point: `IterativeImputer` runs the whole MICE loop; it is still experimental, so it needs an extra import first.

Extra: The rest of this section goes beyond the hand calculation: it covers the scikit-learn class, its settings, and a test on the full 50-row data.

8.1 Switching on the experimental class

Key point: from `sklearn.experimental` import `enable_iterative_imputer` must run before `IterativeImputer` can be imported.

scikit-learn marks `IterativeImputer` as **experimental**: its settings may still change without a deprecation cycle between versions. As of scikit-learn 1.9, importing it directly raises an error (scikit-learn docs, `IterativeImputer`).

Python: The extra import switches the class on.

```
# must come first: switches on the experimental class
```

```
from sklearn.experimental import enable_iterative_imputer
```

```
from sklearn.impute import IterativeImputer
```

The first line looks unused, but importing it is what makes the second line work.

8.2 The main parameters

Key point: The defaults are a `BayesianRidge` model, a mean fill to start, up to 10 iterations, and a stop when the changes are tiny.

Parameter	Meaning	Default
<code>estimator</code>	the regression model trained for each feature	<code>BayesianRidge()</code>

Parameter	Meaning	Default
<code>initial_strategy</code>	the step-0 fill: "mean", "median", "most_frequent" or "constant"	"mean"
<code>max_iter</code>	the largest number of iterations	10
<code>tol</code>	stop when the changes between two iterations fall below <code>tol</code> times the largest value in the data	0.001
<code>imputation_order</code>	the order of the features; "ascending" starts with the feature with the fewest gaps	"ascending"
<code>n_nearest_features</code>	use only this many other features as inputs (faster on wide data)	None (all)
<code>sample_posterior</code>	draw each fill at random from the model's spread instead of its best guess	False
<code>add_indicator</code>	also add a 0/1 column marking each gap (Note 38)	False
<code>random_state</code>	seed for the random parts	None

After fitting, `n_iter_` holds the number of iterations actually run. On the toy table, the default imputer stops after 4.

8.3 Reproducing the hand calculation

Key point: With `estimator=LinearRegression()`, `IterativeImputer` gives exactly the full-precision numbers of Section 7.

Python: One iteration, then ten.

```
from sklearn.linear_model import LinearRegression
imp = IterativeImputer(estimator=LinearRegression(),
                       max_iter=1, tol=0)
```

```
imp.fit_transform(df)
```

```
# gaps: 23.14, 11.06, 31.60
```

`df` is the 5-row table with `np.nan` in the gaps. `tol=0` turns off the early stop, so exactly `max_iter` iterations run. With `max_iter=10` the gaps become 26.72, 13.02 and 70.69. scikit-learn warns that the stopping rule was not reached; here that is on purpose.

8.4 Fitting on the training set only

Key point: Fit the imputer on the training set, then use it to fill both the training and the test set.

As with every imputer, the models must learn only from training observations. If test observations took part in `fit`, information about the test set would leak into training.

Python: Fit on training observations, fill test observations.

```
imp = IterativeImputer(random_state=0)
X_train_filled = imp.fit_transform(X_train)
X_test_filled = imp.transform(X_test)
```

On the full 50-row data, we split 70/30, hid 20% of the values in both parts at random, and filled the test part. The error is the root mean squared difference from the true values, in units of 10,000 dollars, over 100 random splits:

Imputer	Error
Mean (<code>SimpleImputer</code>)	7.54
KNN (<code>KNNImputer</code> , $k = 5$)	6.69
Iterative (<code>IterativeImputer</code>)	5.88

The iterative imputer comes closest to the hidden values, because the features are related: R&D and Marketing have a correlation of 0.72, so each helps to predict the other. Gaps that can be predicted from other features are the MAR case of Section 2. The Extra below tests this reason directly.

Extra: The test behind this (the last cell of the Notebook). We shuffle each column on its own, which keeps every feature's values but breaks the links between features (all correlations fall below 0.2), and rerun the same 100 splits.

Imputer	Error, real data	Error, shuffled columns
Mean	7.54	7.78
KNN, $k = 5$	6.69	8.64
Iterative	5.88	8.10

Without the links between features, the iterative imputer loses its lead and does worse than the plain mean. The lead on real data therefore comes from those links.

8.5 Several imputations

Key point: With `sample_posterior=True`, each seed gives a different, equally plausible filled table; the “multiple” in MICE.

Extra: In statistics, MICE originally means making several filled copies of the data, analysing each one, and combining the results (Rubin 1987; van Buuren and Groothuis-Oudshoorn 2011). The spread between the copies shows how unsure the fills are. With `sample_posterior=True` and a different `random_state` each time, `IterativeImputer` produces such copies. With the default `False`, it gives one best-guess table, which is what a machine learning pipeline usually needs.

9. Summary

	Mean imputation	KNN imputer	Iterative imputer (MICE)
Kind	univariate	multivariate	multivariate
Fill value	feature mean	mean of the k nearest observations	prediction of a model trained on the other features
Runs	once	once	repeated until the fills settle
Best for	MCAR, few gaps	similar observations exist	MAR: gaps predictable from other features
Speed	instant	a distance to every training observation	one model per feature per iteration
Error on 50 Startups scikit-learn	7.54 <code>SimpleImputer</code>	6.69 <code>KNNImputer</code>	5.88 <code>IterativeImputer</code> (experimental import)

- MICE starts with a mean fill (iteration 0).
- In each iteration, every feature in turn gets its gaps back to NaN, a model trained on the observations without a gap, and new predictions for its gaps.
- Each model uses the latest fills of the other features: the equations are chained.
- After each iteration, we subtract the previous table; when the changes are near 0, or after a fixed number of iterations, we stop.
- MICE works best when data is MAR. MICE is accurate but slow, and needs its fitted models in production.
- `IterativeImputer` needs from `sklearn.experimental import enable_iterative_imputer`; defaults are `BayesianRidge`, mean start, `max_iter=10`, `tol=0.001`.
- Fit on the training set only, then transform both sets.

10. Sources

- Rubin, D. B. (1987). *Multiple Imputation for Nonresponse in Surveys*. Wiley.
- van Buuren, S. and Groothuis-Oudshoorn, K. (2011). mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software* 45(3), 1–67.
- scikit-learn API reference, `IterativeImputer`; User Guide, *Iterative imputation and Multiple vs. Single Imputation*.

11. Key terms

Term	Meaning
Feature	An input variable: one column of the data table
Observation	One record: one row of the data table
Iterative imputer	A multivariate imputer that predicts each feature's gaps from the other features, repeating until the fills settle
MICE	Multivariate Imputation by Chained Equations: the algorithm behind the iterative imputer
Chained equations	One prediction model per feature, each using the latest fills of the others
Iteration (MICE)	One pass that re-predicts the gaps of every feature once, in order
Iteration 0	The starting table, with every gap filled by its feature's mean
Convergence	The point where the fills hardly change between two iterations
<code>IterativeImputer</code>	scikit-learn's class for MICE; still experimental
<code>enable_iterative_imputer</code>	The import that switches on the experimental <code>IterativeImputer</code>
<code>BayesianRidge</code>	A linear regression that pulls its weights toward 0 a little; the default model of <code>IterativeImputer</code>
<code>max_iter</code>	The largest number of iterations <code>IterativeImputer</code> runs; default 10
<code>tol</code>	The size of change below which <code>IterativeImputer</code> stops early; default 0.001
<code>sample_posterior</code>	Draw each fill at random from the model's spread, giving several plausible filled tables
Multiple imputation	Making several filled copies of the data to see how unsure the fills are